Data Analyst Capstone Project

IBM Skills Network | Data Analyst Professional Certificate

Table of Contents

Lab 1 - Python Basics

Review Of Accessing APIs

Estimated time needed: 30 minutes

Objectives

After completing this lab, you will be able to:

- Understand HTTP
- Analyze HTTP Requests

Table of Contents Overview of HTTP Uniform Resource Locator: URL Request Response Requests in Python Get Request with URL Parameters Post Requests

Overview of HTTP

When you, the client, access a web page, your browser sends an HTTP request to the server where the page is hosted. The server tries to locate the desired resource by default " index.html ". If your request is successful, the server will send the object to the client in an HTTP response, which includes information like the type of the resource, the length of the resource, and other information. The figure below represents the process. The circle on the left represents the client, the circle on the right represents the web server. The table under the web server represents a list of resources stored in the web server. In this case an HTML file, png image, and txt file . The HTTP protocol enables you to send and receive information through the web including webpages, images, and other web resources. In this lab, you will explore the Requests library for interacting with the HTTP protocol. </p

Uniform Resource Locator (URL)

Uniform resource locator (URL) is the common way to find resources on the web. A URL can be broken down into three main parts: Scheme : The protocol used, which in this lab will always be http:// Internet address or Base URL : Used to locate the resource. Examples include www.ibm.com and www.gitlab.com Route : Location on the web server for example: /images/IDSNlogo.png

You may also hear the term Uniform Resource Identifier (URI). URL are actually a subset of URIs. Another popular term is endpoint, which refers to the URL of an operation provided by a web server.

Request

The process can be broken into the Request and Response process. The request using the get method is partially illustrated below. In the start line we have the GET method, this is an HTTP method. Also the location of the resource /index.html and the HTTP version. The Request header passes additional information with an HTTP request:

When an HTTP request is made, an HTTP method is sent, this tells the server what action to perform. A list of several HTTP methods is shown below. We will review more examples later.

Response

The figure below represents the response; the response start line contains the version number HTTP/1.0 , a status code (200) meaning success, followed by a descriptive phrase (OK). The response header contains useful information. Finally, we have the response body containing the requested file, an HTML document. It should be noted that some requests have headers.

Some status code examples are shown in the table below, the prefix indicates the class. These are shown in yellow, with actual status codes shown in white. Check out the following link for more descriptions.

Install the required libraries

```
!pip install requests
!pip install pillow
```

Requests in Python

Requests is a Python Library that allows you to send HTTP/1.1 requests easily. We can import the library as follows:

```
import requests
```

We will also use the following libraries:

```
import os
from PIL import Image
from IPython.display import IFrame
```

You can make a GET request via the method get to www.ibm.com:

```
url='https://www.ibm.com/'
r=requests.get(url)
```

We have the response object r , this has information about the request, like the status of the request. You can view the status code using the attribute status_code .

```
r.status_code
```

You can view the request headers:

```
print(r.request.headers)
```

You can view the request body, in the following line, as there is nobody for a get request we get None :

```
print("request body:", r.request.body)
```

You can view the HTTP response header using the attribute headers . This returns a python dictionary of HTTP response headers.

```
header=r.headers
print(r.headers)
```

You can obtain the date the request was sent using the key Date .

```
header['date']
```

Content-Type indicates the type of data:

```
header['Content-Type']
```

You can also check the encoding :

```
 r.encoding
```

As the Content-Type is text/html you can use the attribute text to display the HTML in the body. You can review the first 100 characters:

```
r.text[0:100]
```

You can load other types of data for non-text requests, like images. Consider the URL of the following image:

```
# Use single quotation marks for defining string
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0101EN-SkillsNetwo
```

You can make a get request:

```
r=requests.get(url)
```

You can look at the response header:

```
print(r.headers)
```

You can see the 'Content-Type'

```
r.headers['Content-Type']
```

An image is a response object that contains the image as a bytes-like object . As a result, we must save it using a file object. First, you specify the file path and name

```
path=os.path.join(os.getcwd(),'image.png')
```

You save the file, in order to access the body of the response we use the attribute content then save it using the open function and write method :

```
with open(path,'wb') as f:
    f.write(r.content)
```

You can view the image:

```
Image.open(path)
```

Question: Download a file

Consider the following URL:

URL =

Write the commands to download the txt file in the given link.

```
## Write your code here
```

Click here for the solution ``python url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0101EN-SkillsNetwork/labs/Module%205/data/Example1.txt' path=os.path.join(os.getcwd(),'example1.txt') r=requests.get(url) with open(path,'wb') as f: f.write(r.content)``

Get Request with URL Parameters

You can use the GET method to modify the results of your query, for example, retrieving data from an API. We send a GET request to the server. As before, we have the Base URL , in the Route we append /get , this indicates we would like to preform a GET request. This is demonstrated in the following table:

The Base URL is for http://httpbin.org/ . It is a simple HTTP Request and Response service. The URL in Python is given by:

```
url_get='http://httpbin.org/get'
```

A query string is a part of a uniform resource locator (URL), it sends other information to the web server. The start of the query is a ? , followed by a series of parameter and value pairs, as shown in the table below. The first parameter name is name and the value is Joseph . The second parameter name is ID and the Value is 123 . Each pair, parameter, and value is separated by an equals sign, = . The series of pairs is separated by the ampersand & .

To create a Query string, add a dictionary. The keys are the parameter names and the values are the value of the Query string.

```
payload={"name":"Joseph","ID":"123"}
```

Then passing the dictionary payload to the params parameter of the get() function:

```
r=requests.get(url_get,params=payload)
```

You can print out the URL and see the name and values.

```
r.url
```

There is no request body.

```
print("request body:", r.request.body)
```

You can print out the status code.

```
print(r.status_code)
```

You can view the response as text:

```
print(r.text)
```

You can look at the 'Content-Type' .

```
r.headers['Content-Type']
```

As the content 'Content-Type' is in the JSON format, you can use the method json() , it returns a Python dict :

```
r.json()
```

The key args has the name and values:

```
r.json()['args']
```

Post Requests

Like a GET request, a POST is used to send data to a server, but the POST request sends the data in a request body. In order to send the Post Request in Python, in the URL you can change the route to POST :

```
url_post='http://httpbin.org/post'
```

This endpoint will expect data as a file or as a form. A form is convenient way to configure an HTTP request to send data to a server.

To make a POST request we use the post() function, the variable payload is passed to the parameter data :

```
try:
    response = requests.post(url_post, data=payload)
    if response.status_code == 200:
        print("Response JSON:", response.json())
    else:
        print(f"HTTP Error: {response.status_code} - {response.reason}")
except requests.exceptions.RequestException as e:
    print(f"An error occurred: {e}")
```

Comparing the URL from the response object of the GET and POST request you see the POST request has no name or value pairs.

```
print("POST request URL:",response.url )
print("GET request URL:",r.url)
```

You can compare the POST and GET request body, you see only the POST request has a body:

```
print("POST request body:",response.request.body)
print("GET request body:",r.request.body)
```

You can view the form as well:

```
response.json()['form']
```

There is a lot more you can do. Check out Requests for more.

Congratulations, you have completed your hands-on lab on Review Of Accessing APIs.

Authors

Joseph Santarcangelo

Other Contributors

Mavis Zhou

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-04 2.5 Madhusudan ID reviewed

2024-05-23 2.5 Madhusudan ID reviewed

2023-11-02 2.4 Abhishek Gagneja Updated instructions

2023-06-07 2.3 Akansha Yadav Spell Check

2021-12-20 2.1 Malika Updated the links

2020-09-02 2.0 Simran Template updates to the file

© IBM Corporation. All rights reserved.

--!>

Lab 2 - Collecting Jobs Data Using API

Collecting Job Data Using APIs

Estimated time needed: 45 to 60 minutes

Objectives

After completing this lab, you will be able to:

- Collect job data from Jobs API
- Store the collected data into an excel spreadsheet.

> Note: Before starting with the assignment make sure to read all the instructions and then move ahead with the coding part.

Instructions

To run the actual lab, firstly you need to click on the Jobs_API notebook link. The file contains flask code which is required to run the Jobs API data.

Now, to run the code in the file that opens up follow the below steps.

Step1: Download the file.

Step2: Upload it on the IBM Watson studio. (If IBM Watson Cloud service does not work in your system, follow the alternate Step 2 below)

Step2(alternate): Upload it in your SN labs environment using the upload button which is highlighted in red in the image below: Remember to upload this Jobs_API file in the same folder as your current .ipynb file

Step3: Run all the cells of the Jobs_API file. (Even if you receive an asterik sign after running the last cell, the code works fine.)

If you want to learn more about flask, which is optional, you can click on this link here.

Once you run the flask code, you can start with your assignment.

Dataset Used in this Assignment

The dataset used in this lab comes from the following source: https://www.kaggle.com/promptcloud/jobs-on-naukricom under the under a Public Domain license.

> Note: We are using a modified subset of that dataset for the lab, so to follow the lab instructions successfully please use the dataset provided with the lab, rather than the dataset from the original source.

The original dataset is a csv. We have converted the csv to json as per the requirement of the lab.

Warm-Up Exercise

Before you attempt the actual lab, here is a fully solved warmup exercise that will help you to learn how to access an API.

Using an API, let us find out who currently are on the International Space Station (ISS). The API at http://api.open-notify.org/astros.json gives us the information of astronauts currently on ISS in json format. You can read more about this API at http://open-notify.org/Open-Notify-API/People-In-Space/

```
import requests # you need this module to make an API call
import pandas as pd

api_url = "http://api.open-notify.org/astros.json" # this url gives use the astronaut data

response = requests.get(api_url) # Call the API using the get method and store the
                                 # output of the API call in a variable called response.

if response.ok:              # if all is well() no errors, no network timeouts)
    data = response.json()  # store the result in json format in a variable called data
                            # the variable data is of type dictionary.

print(data)   # print the data just to check the output or for debugging
```

Print the number of astronauts currently on ISS.

```
print(data.get('number'))
```

Print the names of the astronauts currently on ISS.

```
astronauts = data.get('people')
print("There are {} astronauts on ISS".format(len(astronauts)))
print("And their names are :")
for astronaut in astronauts:
    print(astronaut.get('name'))
```

Hope the warmup was helpful. Good luck with your next lab!

Lab: Collect Jobs Data using Jobs API

Objective: Determine the number of jobs currently open for various technologies and for various locations

Collect the number of job postings for the following locations using the API:

- Los Angeles
- New York
- San Francisco
- Washington DC
- Seattle
- Austin
- Detroit

```
#Import required libraries
import pandas as pd
import json
```

Write a function to get the number of jobs for the Python technology. > Note: While using the lab you need to pass the payload information for the params attribute in the form of key value pairs. Refer the ungraded rest api lab in the course Python for Data Science, AI & Development link ##### The keys in the json are Job Title Job Experience Required Key Skills Role Category Location Functional Area Industry Role You can also view the json file contents from the following json URL.

```
api_url="http://127.0.0.1:5000/data"
def get_number_of_jobs_T(technology):

    #your code goes here
    return technology,number_of_jobs
```

Calling the function for Python and checking if it works.

```
get_number_of_jobs_T("Python")
```

Write a function to find number of jobs in US for a location of your choice

```
def get_number_of_jobs_L(location):

    #your coe goes here
    return location,number_of_jobs
```

Call the function for Los Angeles and check if it is working.

```
#your code goes here
```

Store the results in an excel file

Call the API for all the given technologies above and write the results in an excel spreadsheet.

If you do not know how create excel file using python, double click here for hints.

<!--

from openpyxl import Workbook # import Workbook class from module openpyxl wb=Workbook() # create a workbook object ws=wb.active # use the active worksheet ws.append(['Country','Continent']) # add a row with two columns 'Country' and 'Continent' ws.append(['Eygpt','Africa']) # add a row with two columns 'Egypt' and 'Africa' ws.append(['India','Asia']) # add another row ws.append(['France','Europe']) # add another row wb.save("countries.xlsx") # save the workbook into a file called countries.xlsx

-->

Create a python list of all locations for which you need to find the number of jobs postings.

```
#your code goes here
```

Import libraries required to create excel spreadsheet

```
# your code goes here
```

Create a workbook and select the active worksheet

```
# your code goes here
```

Find the number of jobs postings for each of the location in the above list. Write the Location name and the number of jobs postings into the excel spreadsheet.

```
#your code goes here
```

Save into an excel spreadsheet named 'job-postings.xlsx'.

```
#your code goes here
```

In the similar way, you can try for below given technologies and results can be stored in an excel sheet.

Collect the number of job postings for the following languages using the API:

- C
- C#
- C++
- Java
- JavaScript
- Python
- Scala
- Oracle
- SQL Server
- MySQL Server
- PostgreSQL
- MongoDB

```
# your code goes here
```

Author

Ayushi Jain

Other Contributors

Rav Ahuja

Lakshmi Holla

Malika

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2022-01-19 0.3 Lakshmi Holla Added changes in the markdown

2021-06-25 0.2 Malika Updated GitHub job json link

2020-10-17 0.1 Ramesh Sannareddy Created initial version of the lab

---

Lab 3 - Web Scraping Review

Web Scraping Lab

Estimated time needed: 30 minutes

Objectives

After completing this lab, you will be able to:

- Download a webpage using requests module
- Scrape all links from a webpage

- Scrape all image URLs from a web page
- Scrape data from html tables

Scrape www.ibm.com

Import the required modules and functions

```
from bs4 import BeautifulSoup # this module helps in web scrapping.
import requests  # this module helps us to download a webpage
```

Download the contents of the webpage

```
url = "http://www.ibm.com"
```

```
# get the contents of the webpage in text format and store in a variable called data
data  = requests.get(url).text
```

Create a soup object using the class BeautifulSoup

```
soup = BeautifulSoup(data,"html.parser")  # create a soup object using the variable 'data'
```

Scrape all links

```
for link in soup.find_all('a'):  # in html anchor/link is represented by the tag &lt;a&gt;
    print(link.get('href'))
```

Scrape all images

```
for link in soup.find_all('img'):# in html image is represented by the tag &lt;img&gt;
    print(link.get('src'))
```

Scrape data from html tables

```
#The below URL contains a html table with data about colors and color codes.
URL = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DA0321EN-SkillsNetwork/labs/datasets/HTM
```

Before proceeding to scrape a website, you need to examine the contents, and the way data is organized on the website. Open the above URL in your browser and check how many rows and columns are there in the color table.

```
# get the contents of the webpage in text format and store in a variable called data
data  = requests.get(URL).text
```

```
soup = BeautifulSoup(data,"html.parser")
```

```
#find a html table in the web page
table = soup.find('table') # in html table is represented by the tag &lt;table&gt;
```

Get all rows from the table

```
for row in table.find_all('tr'): # in html table row is represented by the tag &lt;tr&gt;
    # Get all columns in each row.
    cols = row.find_all('td') # in html a column is represented by the tag &lt;td&gt;
    color_name = cols[2].getText() # store the value in column 3 as color_name
    color_code = cols[3].getText() # store the value in column 4 as color_code
    print("{}---&gt;{}".format(color_name,color_code))
```

Authors

Ramesh Sannareddy

Other Contributors

Rav Ahuja

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-29 0.2 Madhusudhan Moole Updated lab

2020-10-17 0.1 Ramesh Sannareddy Created initial version of the lab

--!>

Lab 4 - Web Scraping

Hands-on Lab : Web Scraping

Estimated time needed: 30 to 45 minutes

Objectives

In this lab you will perform the following:

- Extract information from a given web site
- Write the scraped data into a csv file.

Extract information from the given web site

You will extract the data from the below web site:

```
#this url contains the data you need to scrape
url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DA0321EN-SkillsNetwork/labs/datasets/Pro
```

The data you need to scrape is the name of the programming language and average annual salary. It is a good idea to open the url in your web broswer and study the contents of the web page before you start to scrape.

Import the required libraries

```
# Your code here
```

Download the webpage at the url

```
#your code goes here
```

Create a soup object

```
#your code goes here
```

Scrape the Language name and annual average salary.

```
#your code goes here
```

Save the scrapped data into a file named popular-languages.csv

```
# your code goes here
```

Authors

Ramesh Sannareddy

Other Contributors

Rav Ahuja

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2020-10-17 0.1 Ramesh Sannareddy Created initial version of the lab

---

Lab 5 - Exploring the Dataset

Lab: Exploring the Dataset

Estimated time needed: 30 minutes

Introduction

Data exploration is the initial phase of data analysis where we aim to understand the data's characteristics, identify patterns, and uncover potential insights. It is a crucial step that helps us make informed decisions about subsequent analysis.

Objectives

After completing this lab, you will be able to:

- Summarize the key characteristics of a dataset.
- Identify different data types commonly used in data analysis.

Install the required library

```
!pip install pandas
```

```
import pandas as pd
```

Load the dataset

Read Data We utilize the pandas.read_csv() function for reading CSV files.

The functions below will download the dataset into your browser:

```
# Load the dataset directly from the URL
file_path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/VYPrOu0Vs3I0hKLLjiPGrA/survey-data-wit
```

Utilize the Pandas method read_csv() to load the data into a dataframe.

```
df = pd.read_csv(file_path)
```

Hands on Lab

Explore the dataset

It is a good idea to print the top 5 rows of the dataset to get a feel of how the dataset will look.

Display the top 5 rows and columns from your dataset.

```
## Write your code here
```

Find out the number of rows and columns

Start by exploring the numbers of rows and columns of data in the dataset.

Print the number of rows in the dataset.

```
## Write your code here
```

Print the number of columns in the dataset.

```
## Write your code here
```

Identify the data types of each column

Explore the dataset and identify the data types of each column.

Print the datatype of all columns.

```
## Write your code here
```

Print the mean age of the survey participants.

First let's map age ranges to approximate numeric values

```
age_mapping = {
    "Under 18 years old": 17,
```

```
    "18-24 years old": 21,
    "25-34 years old": 29,
    "35-44 years old": 39,
    "45-54 years old": 49,
    "55-64 years old": 59,
    "65 years or older": 70
}

# Create a new numeric Age column
df['Age_numeric'] = df['Age'].map(age_mapping)

## Write your code here
# Calculate the mean age
```

The dataset is the result of a world wide survey. Print how many unique countries are there in the Country column.

```
## Write your code here
```

Lab 6 - Finding Duplicates

Finding Duplicates Lab

Estimated time needed: 30 minutes

Introduction

Data wrangling is a critical step in preparing datasets for analysis, and handling duplicates plays a key role in ensuring data accuracy. In this lab, you will focus on identifying and removing duplicate entries from your dataset.

Objectives

In this lab, you will perform the following:

1. Identify duplicate rows in the dataset and analyze their characteristics.
2. Visualize the distribution of duplicates based on key attributes.
3. Remove duplicate values strategically based on specific criteria.
4. Outline the process of verifying and documenting duplicate removal.

Hands on Lab

Install the needed library

```
!pip install pandas
!pip install matplotlib
```

Import pandas module

```
import pandas as pd
```

Import matplotlib

```
import matplotlib.pyplot as plt
```

Load the dataset into a dataframe

Read Data We utilize the pandas.read_csv() function for reading CSV files. However, in this version of the lab, which operates on JupyterLite, the dataset needs to be downloaded to the interface using the provided code below.

```
# Load the dataset directly from the URL
file_path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/VYPrOu0Vs3I0hKLLjiPGrA/survey-data-wit
df = pd.read_csv(file_path)

# Display the first few rows
print(df.head())
```

Load the data into a pandas dataframe:

Note: If you are working on a local Jupyter environment, you can use the URL directly in the pandas.read_csv() function as shown below:

```
# df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-d
```

Identify and Analyze Duplicates

Task 1: Identify Duplicate Rows

1. Count the number of duplicate rows in the dataset.
3. Display the first few duplicate rows to understand their structure.

```
## Write your code here
```

Task 2: Analysis of Duplicate Response Patterns

1. Identify duplicate response patterns based on selected columns such as MainBranch, Employment, and RemoteWork.
2. Clarify that these represent multiple respondents with identical answers rather than duplicate records. Analyse which other columns frequently share identical values within these response-pattern groups.

```
## Write your code here
```

Task 3: Visualize Shared Response Patterns

1. Create visualizations to show the distribution of shared response patterns across different categories.
2. Use bar charts or pie charts to represent the distribution of respondents who share identical values for MainBranch, Employment, and RemoteWork, grouped by Country and Employment.

```
## Write your code here
```

Task 4: Evaluate Duplicate Handling Strategy

1. Analyse the dataset to determine which column(s) define record uniqueness.
2. Assess whether removing rows based on a subset of columns (rather than complete row duplication) is appropriate.

Justify your decision with reference to the structure and purpose of the dataset.

Verify and Document Duplicate Removal Process

Task 5: Documentation

1. Document the process of identifying and removing duplicates.
2. Explain the reasoning behind selecting specific columns for identifying and removing duplicates.

Summary and Next Steps

In this lab, you focused on identifying and analyzing duplicate rows within the dataset.

• You employed various techniques to explore the nature of duplicates and applied strategic methods for their removal.

• For additional analysis, consider investigating the impact of duplicates on specific analyses and how their removal affects the results.

• This version of the lab is more focused on duplicate analysis and handling, providing a structured approach to deal with duplicates in a dataset effectively.

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-11- 05 1.3 Madhusudhan Moole Updated lab

2024-10-28 1.2 Madhusudhan Moole Updated lab

2024-09-24 1.1 Madhusudhan Moole Updated lab

2024-09-23 1.0 Raghul Ramesh Created lab

--!>

Lab 7 - Removing Duplicates

Removing Duplicates

Estimated time needed: 45 to 60 minutes

Introduction

In this lab, you will focus on data wrangling, an important step in preparing data for analysis. Data wrangling involves cleaning and organizing data to make it suitable for analysis. One key task in this process is removing duplicate entries, which are repeated entries that can distort analysis and lead to inaccurate conclusions.

Objectives

In this lab you will perform the following:

1. Identify duplicate rows in the dataset.
2. Use suitable techniques to remove duplicate rows and verify the removal.
3. Summarize how to handle missing values appropriately.
4. Use ConvertedCompYearly to normalize compensation data.

Install the Required Libraries

```
!pip install pandas
```

Step 1: Import Required Libraries

```
import pandas as pd
```

Step 2: Load the Dataset into a DataFrame

Read Data

If you are using JupyterLite, use the code below to download the dataset into your environment. If you are using a local environment, you can use the direct URL with pd.read_csv() .

```
from pyodide.http import pyfetch

async def download(url, filename):
    response = await pyfetch(url)
    if response.status == 200:
        with open(filename, "wb") as f:
            f.write(await response.bytes())

# Define the file path for the data
file_path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv

# Download the dataset
await download(file_path, "survey_data.csv")
file_name = "survey_data.csv"
```

Load the data into a pandas dataframe:

```
df = pd.read_csv(file_name)
```

Note: If you are working on a local Jupyter environment, you can use the URL directly in the pandas.read_csv() function as shown below:

df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv")

Step 3: Identifying Duplicate Rows

Task 1: Identify Duplicate Rows 1. Count the number of duplicate rows in the dataset. 2. Display the first few duplicate rows to understand their structure.

```
# your code goes here
```

Step 4: Removing Duplicate Rows

Task 2: Remove Duplicates 1. Remove duplicate rows from the dataset using the drop_duplicates() function.

2. Verify the removal by counting the number of duplicate rows after removal .

```
# your code goes here
```

Step 5: Handling Missing Values

Task 3: Identify and Handle Missing Values 1. Identify missing values for all columns in the dataset. 2. Choose a column with significant missing values (e.g., EdLevel) and impute with the most frequent value.

```
# your code goes here
```

Step 6: Normalizing Compensation Data

Task 4: Normalize Compensation Data Using ConvertedCompYearly 1. Use the ConvertedCompYearly column for compensation analysis as the normalized annual compensation is already provided. 2. Check for missing values in ConvertedCompYearly and handle them if necessary.

```
# your code goes here
```

Step 7: Summary and Next Steps

In this lab, you focused on identifying and removing duplicate rows.

- You handled missing values by imputing the most frequent value in a chosen column.
- You used ConvertedCompYearly for compensation normalization and handled missing values.
- For further analysis, consider exploring other columns or visualizing the cleaned dataset.

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-11-05 1.2 Madhusudhan Moole Updated lab

2024-09-24 1.1 Madhusudhan Moole Updated lab

2024-09-23 1.0 Raghul Ramesh Created lab

--!>

Lab 8 - Finding Missing Values

Finding Missing Values

Estimated time needed: 30 minutes

Data wrangling is the process of cleaning, transforming, and organizing data to make it suitable for analysis. Finding and handling missing values is a crucial step in this process to ensure data accuracy and completeness. In this lab, you will focus exclusively on identifying and handling missing values in the dataset.

Objectives

After completing this lab, you will be able to:

- Identify missing values in the dataset.
- Quantify missing values for specific columns.
- Impute missing values using various strategies.

Hands on Lab

Setup: Install Required Libraries

```
!pip install pandas
!pip install matplotlib
!pip install seaborn
```

Import Necessary Modules:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Tasks

1. Load the Dataset We use the pandas.read_csv() function for reading CSV files. However, in this version of the lab, which operates on JupyterLite, the dataset needs to be downloaded to the interface using the provided code below.

The functions below will download the dataset into your browser:

```
# Define the URL of the dataset
file_path ="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/UDKAZw-kz18Yj8P6icf_qw/survey-data-dupl

# Load the dataset into a DataFrame
df = pd.read_csv(file_path)

# Display the first few rows to ensure it loaded correctly
print(df.head())
```

2. Explore the Dataset

Task 1: Display basic information and summary statistics of the dataset.

```
## Write your code here
```

3. Finding Missing Values

Task 2: Identify missing values for all columns.

```
## Write your code here
```

Task 3: Visualize missing values using a heatmap (Using seaborn library).

```
## Write your code here
```

Task 4: Count the number of missing rows for a specific column (e.g., Employment).

```
## Write your code here
```

4. Imputing Missing Values

Task 5: Identify the most frequent (majority) value in a specific column (e.g., Employment).

```
## Write your code here
```

Task 6: Impute missing values in the Employment column with the most frequent value.

```
## Write your code here
```

5. Visualizing Imputed Data

Task 7: Visualize the distribution of a column after imputation (e.g., Employment).

```
## Write your code here
```

Summary

In this lab, you:

- Loaded the dataset into a pandas DataFrame.
- Identified missing values across all columns.
- Quantified missing values in specific columns.
- Imputed missing values in a categorical column using the most frequent value.
- Visualized the imputed data for better understanding.

Lab 9 - Imputing Missing Values

Impute Missing Values

Estimated time needed: 30 minutes

In this lab, you will practice essential data wrangling techniques using the Stack Overflow survey dataset. The primary focus is on handling missing data and ensuring data quality. You will:

- Load the Data: Import the dataset into a DataFrame using the pandas library.
- Clean the Data: Identify and remove duplicate entries to maintain data integrity.
- Handle Missing Values: Detect missing values, impute them with appropriate strategies, and verify the imputation to create a complete and reliable dataset for analysis.

This lab equips you with the skills to effectively preprocess and clean real-world datasets, a crucial step in any data analysis project.

Objectives

In this lab, you will perform the following:

- Identify missing values in the dataset.
- Apply techniques to impute missing values in the dataset.
- Use suitable techniques to normalize data in the dataset.

Install needed library

```
!pip install pandas
```

Step 1: Import Required Libraries

```
import pandas as pd
```

Step 2: Load the Dataset Into a Dataframe

Read Data

The functions below will download the dataset into your browser:

```
file_path ="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/UDKAZw-kz18Yj8P6icf_qw/survey-data-dupl
df = pd.read_csv(file_path)

# Display the first few rows to ensure it loaded correctly
print(df.head())
```

Step 3. Finding and Removing Duplicates

Task 1: Identify duplicate rows in the dataset.

```
## Write your code here
```

Task 2: Remove the duplicate rows from the dataframe.

```
## Write your code here
```

Step 4: Finding Missing Values

Task 3: Find the missing values for all columns.

```
## Write your code here
```

Task 4: Find out how many rows are missing in the column RemoteWork.

```
## Write your code here
```

Step 5. Imputing Missing Values

Task 5: Find the value counts for the column RemoteWork.

```
## Write your code here
```

Task 6: Identify the most frequent (majority) value in the RemoteWork column.

```
## Write your code here
```

Task 7: Impute (replace) all the empty rows in the column RemoteWork with the majority value.

```
## Write your code here
```

Task 8: Check for any compensation-related columns and describe their distribution.

```
## Write your code here
```

Summary

In this lab, you focused on imputing missing values in the dataset.

- Use the pandas.read_csv() function to load a dataset from a CSV file into a DataFrame.
- Download the dataset if it's not available online and specify the correct file path.

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-11-05 1.3 Madhusudhan Moole Updated lab

2024-10-29 1.2 Madhusudhan Moole Updated lab

2024-09-27 1.1 Madhusudhan Moole Updated lab

2024-09-26 1.0 Raghul Ramesh Created lab

--!>

Lab 10 - Normalizing Data

Data Normalization Techniques

Estimated time needed: 30 minutes

In this lab, you will focus on data normalization. This includes identifying compensation-related columns, applying normalization techniques, and visualizing the data distributions.

Objectives

In this lab, you will perform the following:

- Identify duplicate rows and remove them.
- Check and handle missing values in key columns.
- Identify and normalize compensation-related columns.
- Visualize the effect of normalization techniques on data distributions.

Hands on Lab

Step 1: Install and Import Libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load the Dataset into a DataFrame

We use the pandas.read_csv() function for reading CSV files. However, in this version of the lab, which operates on JupyterLite, the dataset needs to be downloaded to the interface using the provided code below.

The functions below will download the dataset into your browser:

```
file_path ="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/UDKAZw-kz18Yj8P6icf_qw/survey-data-dupl

df = pd.read_csv(file_path)

# Display the first few rows to check if data is loaded correctly
print(df.head())
```

```
#df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/UDKAZw-kz18Yj8P6icf_qw/survey-da
```

Section 1: Handling Duplicates

Task 1: Identify and remove duplicate rows.

```
## Write your code here
```

Section 2: Handling Missing Values

Task 2: Identify missing values in CodingActivities.

```
## Write your code here
```

Task 3: Impute missing values in CodingActivities with forward-fill.

```
## Write your code here
```

Note: Before normalizing ConvertedCompYearly, ensure that any missing values (NaN) in this column are handled appropriately. You can choose to either drop the rows containing NaN or replace the missing values with a suitable statistic (e.g., median or mean).

Section 3: Normalizing Compensation Data

Task 4: Identify compensation-related columns, such as ConvertedCompYearly.

Normalization is commonly applied to compensation data to bring values within a comparable range. Here, you'll identify ConvertedCompYearly or similar columns, which contain compensation information. This column will be used in the subsequent tasks for normalization.

```
## Write your code here
```

Task 5: Normalize ConvertedCompYearly using Min-Max Scaling.

Min-Max Scaling brings all values in a column to a 0-1 range, making it useful for comparing data across different scales. Here, you will apply Min-Max normalization to the ConvertedCompYearly column, creating a new column ConvertedCompYearly_MinMax with normalized values.

```
## Write your code here
```

Task 6: Apply Z-score Normalization to ConvertedCompYearly.

Z-score normalization standardizes values by converting them to a distribution with a mean of 0 and a standard deviation of 1. This method is helpful for datasets with a Gaussian (normal) distribution. Here, you'll calculate Z-scores for the ConvertedCompYearly column, saving the results in a new column ConvertedCompYearly_Zscore.

```
## Write your code here
```

Section 4: Visualization of Normalized Data

Task 7: Visualize the distribution of ConvertedCompYearly, ConvertedCompYearlyNormalized, and ConvertedCompYearlyZscore

Visualization helps you understand how normalization changes the data distribution. In this task, create histograms for the original ConvertedCompYearly, as well as its normalized versions (ConvertedCompYearlyMinMax and ConvertedCompYearlyZscore). This will help you compare how each normalization technique affects the data range and distribution.

```
## Write your code here
```

Summary

In this lab, you practiced essential normalization techniques, including:

- Identifying and handling duplicate rows.
- Checking for and imputing missing values.
- Applying Min-Max scaling and Z-score normalization to compensation data.
- Visualizing the impact of normalization on data distribution.

Lab 11 - Data Wrangling

Data Wrangling Lab

Estimated time needed: 45 minutes

In this lab, you will perform data wrangling tasks to prepare raw data for analysis. Data wrangling involves cleaning, transforming, and organizing data into a structured format suitable for analysis. This lab focuses on tasks like identifying inconsistencies, encoding categorical variables, and feature transformation.

Objectives

After completing this lab, you will be able to:

- Identify and remove inconsistent data entries.
- Encode categorical variables for analysis.
- Handle missing values using multiple imputation strategies.
- Apply feature scaling and transformation techniques.

Intsall the required libraries

```
!pip install pandas
!pip install matplotlib
```

Tasks

Step 1: Import the necessary module.

1. Load the Dataset

1.1 Import necessary libraries and load the dataset.

Ensure the dataset is loaded correctly by displaying the first few rows.

```
# Import necessary libraries
import pandas as pd

# Load the Stack Overflow survey data
dataset_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.c
df = pd.read_csv(dataset_url)

# Display the first few rows
print(df.head())
```

2. Explore the Dataset

2.1 Summarize the dataset by displaying the column data types, counts, and missing values.

```
# Write your code here
```

2.2 Generate basic statistics for numerical columns.

```
# Write your code here
```

3. Identifying and Removing Inconsistencies

3.1 Identify inconsistent or irrelevant entries in specific columns (e.g., Country).

```
# Write your code here
```

3.2 Standardize entries in columns like Country or EdLevel by mapping inconsistent values to a consistent format.

```
## Write your code here
```

4. Encoding Categorical Variables

4.1 Encode the Employment column using one-hot encoding.

```
## Write your code here
```

5. Handling Missing Values

5.1 Identify columns with the highest number of missing values.

```
## Write your code here
```

5.2 Impute missing values in numerical columns (e.g., ConvertedCompYearly) with the mean or median.

```
## Write your code here
```

5.3 Impute missing values in categorical columns (e.g., RemoteWork) with the most frequent value.

```
## Write your code here
```

6. Feature Scaling and Transformation

6.1 Apply Min-Max Scaling to normalize the ConvertedCompYearly column.

```
## Write your code here
```

6.2 Log-transform the ConvertedCompYearly column to reduce skewness.

```
## Write your code here
```

7. Feature Engineering

7.1 Create a new column ExperienceLevel based on the YearsCodePro column:

```
## Write your code here
```

Summary

In this lab, you:

- Explored the dataset to identify inconsistencies and missing values.
- Encoded categorical variables for analysis.
- Handled missing values using imputation techniques.
- Normalized and transformed numerical data to prepare it for analysis.
- Engineered a new feature to enhance data interpretation.

Lab 12 - Exploratory Data Analysis

Lab: Exploratory Data Analysis

Estimated time needed: 30 minutes

In this lab, you will work with a cleaned dataset to perform Exploratory Data Analysis or EDA.

Objectives

In this lab, you will perform the following:

- Examine the structure of a dataset.
- Handle missing values effectively.
- Conduct summary statistics on key columns.
- Analyze employment status, job satisfaction, programming language usage, and trends in remote work.

Hands on Lab

Step 1: Install and Import Libraries

Install the necessary libraries for data manipulation and visualization.

```
!pip install pandas
!pip install matplotlib
!pip install seaborn

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load and Preview the Dataset

Load the dataset from the provided URL. Use df.head() to display the first few rows to get an overview of the structure.

```
# Load the Stack Overflow survey dataset
data_url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv'
df = pd.read_csv(data_url)

# Set pandas option to display all columns
pd.set_option('display.max_columns', None)

# Display the first few rows of the dataset
df.head()
```

Step 3: Handling Missing Data

Identify and manage missing values in critical columns such as Employment, JobSat, and RemoteWork. Implement a strategy to fill or drop these values, depending on the significance of the missing data.

```
## Write your code here
```

Step 4: Analysis of Experience and Job Satisfaction

Analyze the relationship between years of professional coding experience (YearsCodePro) and job satisfaction (JobSat). Summarize YearsCodePro and calculate median satisfaction scores based on experience ranges.

- Create experience ranges for YearsCodePro (e.g., 0-5, 5-10, 10-20, >20 years).
- Calculate the median JobSat for each range.
- Visualize the relationship using a bar plot or similar visualization.

```
## Write your code here
```

Step 5: Visualize Job Satisfaction

Use a count plot to show the distribution of JobSat values. This provides insights into the overall satisfaction levels of respondents.

```
## Write your code here
```

Step 6: Analyzing Remote Work Preferences by Job Role

Analyze trends in remote work based on job roles. Use the RemoteWork and Employment columns to explore preferences and examine if specific job roles prefer remote work more than others.

- Use a count plot to show remote work distribution.
- Cross-tabulate remote work preferences by employment type (e.g., full-time, part-time) and job roles.

```
## Write your code here
```

Step 7: Analyzing Programming Language Trends by Region

Analyze the popularity of programming languages by region. Use the LanguageHaveWorkedWith column to investigate which languages are most used in different regions.

- Filter data by country or region.
- Visualize the top programming languages by region with a bar plot or heatmap.

```
## Write your code here
```

Step 8: Correlation Between Experience and Satisfaction

Examine how years of experience (YearsCodePro) correlate with job satisfaction (JobSatPoints_1). Use a scatter plot to visualize this relationship.

```
## Write your code here
```

Step 9: Educational Background and Employment Type

Explore how educational background (EdLevel) relates to employment type (Employment). Use cross-tabulation and visualizations to understand if higher education correlates with specific employment types.

```
## Write your code here
```

Step 10: Save the Cleaned and Analyzed Dataset

After your analysis, save the modified dataset for further use or sharing.

```
## Write your code here
```

Summary

In this revised lab, you:

- Loaded and explored the structure of the dataset.
- Handled missing data effectively.
- Analyzed key variables, including working hours, job satisfaction, and remote work trends.
- Investigated programming language usage by region and examined the relationship between experience and satisfaction.
- Used cross-tabulation to understand educational background and employment type.

Lab 13 - Finding How The Data is Distributed

Finding How The Data Is Distributed

Estimated time needed: 30 minutes

In this lab, you will work with a cleaned dataset to perform Exploratory Data Analysis (EDA). You will examine the structure of the data, visualize key variables, and analyze trends related to developer experience, tools, job satisfaction, and other important aspects.

Objectives

In this lab you will perform the following:

- Understand the structure of the dataset.
- Perform summary statistics and data visualization.
- Identify trends in developer experience, tools, job satisfaction, and other key variables.

Install the required libraries

```
!pip install pandas
!pip install matplotlib
!pip install seaborn
```

Step 1: Import Libraries and Load Data

- Import the pandas, matplotlib.pyplot, and seaborn libraries.

• You will begin with loading the dataset. You can use the pyfetch method if working on JupyterLite. Otherwise, you can use pandas' read_csv() function directly on their local machines or cloud environments.

```
# Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the Stack Overflow survey dataset
data_url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv'
df = pd.read_csv(data_url)

# Display the first few rows of the dataset
df.head()
```

Step 2: Examine the Structure of the Data

• Display the column names, data types, and summary information to understand the data structure.

• Objective: Gain insights into the dataset's shape and available variables.

```
## Write your code here
```

Step 3: Handle Missing Data

• Identify missing values in the dataset.

• Impute or remove missing values as necessary to ensure data completeness.

```
## Write your code here
```

Step 4: Analyze Key Columns

• Examine key columns such as Employment, JobSat (Job Satisfaction), and YearsCodePro (Professional Coding Experience).

• Instruction: Calculate the value counts for each column to understand the distribution of responses.

```
## Write your code here
```

Step 5: Visualize Job Satisfaction (Focus on JobSat)

• Create a pie chart or KDE plot to visualize the distribution of JobSat.

• Provide an interpretation of the plot, highlighting key trends in job satisfaction.

```
## Write your code here
```

Step 6: Programming Languages Analysis

• Compare the frequency of programming languages in LanguageHaveWorkedWith and LanguageWantToWorkWith.

• Visualize the overlap or differences using a Venn diagram or a grouped bar chart.

```
## Write your code here
```

Step 7: Analyze Remote Work Trends

• Visualize the distribution of RemoteWork by region using a grouped bar chart or heatmap.

```
## Write your code here
```

Step 8: Correlation between Job Satisfaction and Experience

• Analyze the correlation between overall job satisfaction (JobSat) and YearsCodePro.

• Calculate the Pearson or Spearman correlation coefficient.

```
## Write your code here
```

Step 9: Cross-tabulation Analysis (Employment vs. Education Level)

• Analyze the relationship between employment status (Employment) and education level (EdLevel).

• Instruction: Create a cross-tabulation using pd.crosstab() and visualize it with a stacked bar plot if possible.

```
## Write your code here
```

Step 10: Export Cleaned Data

- Save the cleaned dataset to a new CSV file for further use or sharing.

```
## Write your code here
```

Summary:

In this lab, you practiced key skills in exploratory data analysis, including:

- Examining the structure and content of the Stack Overflow survey dataset to understand its variables and data types.
- Identifying and addressing missing data to ensure the dataset's quality and completeness.
- Summarizing and visualizing key variables such as job satisfaction, programming languages, and remote work trends.
- Analyzing relationships in the data using techniques like:

- Comparing programming languages respondents have worked with versus those they want to work with.

- Exploring remote work preferences by region.

- Investigating correlations between professional coding experience and job satisfaction.
- Performing cross-tabulations to analyze relationships between employment status and education levels.

Authors:

Ayushi Jain

Other Contributors:

Rav Ahuja Lakshmi Holla Malika

Lab 14 - Finding Outliers

Finding Outliers

Estimated time needed: 30 minutes

In this lab, you will work with a cleaned dataset to perform exploratory data analysis or EDA. You will explore the distribution of key variables and focus on identifying outliers in this lab.

Objectives

In this lab, you will perform the following:

- Analyze the distribution of key variables in the dataset.
- Identify and remove outliers using statistical methods.
- Perform relevant statistical and correlation analysis.

Install and import the required libraries

```
!pip install pandas
!pip install matplotlib
!pip install seaborn

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 1: Load and Explore the Dataset

Load the dataset into a DataFrame and examine the structure of the data.

```
file_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv"

#Create the dataframe
df = pd.read_csv(file_url)
```

```
#Display the top 10 records
df.head()
```

Step 2: Plot the Distribution of Industry

Explore how respondents are distributed across different industries.

- Plot a bar chart to visualize the distribution of respondents by industry.
- Highlight any notable trends.

```
##Write your code here
```

Step 3: Identify High Compensation Outliers

Identify respondents with extremely high yearly compensation.

- Calculate basic statistics (mean, median, and standard deviation) for ConvertedCompYearly.
- Identify compensation values exceeding a defined threshold (e.g., 3 standard deviations above the mean).

```
##Write your code here
```

Step 4: Detect Outliers in Compensation

Identify outliers in the ConvertedCompYearly column using the IQR method.

- Calculate the Interquartile Range (IQR).
- Determine the upper and lower bounds for outliers.
- Count and visualize outliers using a box plot.

```
##Write your code here
```

Step 5: Remove Outliers and Create a New DataFrame

Remove outliers from the dataset.

- Create a new DataFrame excluding rows with outliers in ConvertedCompYearly.
- Validate the size of the new DataFrame.

```
##Write your code here
```

Step 6: Correlation Analysis

Analyze the correlation between Age (transformed) and other numerical columns.

- Map the Age column to approximate numeric values.
- Compute correlations between Age and other numeric variables.
- Visualize the correlation matrix.

```
##Write your code here
```

Summary

In this lab, you developed essential skills in Exploratory Data Analysis (EDA) with a focus on outlier detection and removal. Specifically, you:

- Loaded and explored the dataset to understand its structure.
- Analyzed the distribution of respondents across industries.
- Identified and removed high compensation outliers using statistical thresholds and the Interquartile Range (IQR) method.
- Performed correlation analysis, including transforming the Age column into numeric values for better analysis.

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-1 1.1 Madhusudan Moole Reviewed and updated lab

2024-09-29 1.0 Raghul Ramesh Created lab

--!>

Lab 15 - Finding Correlation

Finding Correlation

Estimated time needed: 30 minutes

In this lab, you will work with a cleaned dataset to perform exploratory data analysis (EDA). You will examine the distribution of the data, identify outliers, and determine the correlation between different columns in the dataset.

Objectives

In this lab, you will perform the following:

- Identify the distribution of compensation data in the dataset.
- Remove outliers to refine the dataset.
- Identify correlations between various features in the dataset.

Hands on Lab

Step 1: Install and Import Required Libraries

```
# Install the necessary libraries
!pip install pandas
!pip install matplotlib
!pip install seaborn

# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load the Dataset

```
# Load the dataset from the given URL
file_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv"
df = pd.read_csv(file_url)

# Display the first few rows to understand the structure of the dataset
df.head()
```

Step 3: Analyze and Visualize Compensation Distribution

Task: Plot the distribution and histogram for ConvertedCompYearly to examine the spread of yearly compensation among respondents.

```
## Write your code here
```

Step 4: Calculate Median Compensation for Full-Time Employees

Task: Filter the data to calculate the median compensation for respondents whose employment status is "Employed, full-time."

```
## Write your code here
```

Step 5: Analyzing Compensation Range and Distribution by Country

Explore the range of compensation in the ConvertedCompYearly column by analyzing differences across countries. Use box plots to compare the compensation distributions for each country to identify variations and anomalies within each region, providing insights into global compensation trends.

```
## Write your code here
```

Step 6: Removing Outliers from the Dataset

Task: Create a new DataFrame by removing outliers from the ConvertedCompYearly column to get a refined dataset for correlation analysis.

```
## Write your code here
```

Step 7: Finding Correlations Between Key Variables

Task: Calculate correlations between ConvertedCompYearly, WorkExp, and JobSatPoints_1. Visualize these correlations with a heatmap.

```
## Write your code here
```

Step 8: Scatter Plot for Correlations

Task: Create scatter plots to examine specific correlations between ConvertedCompYearly and WorkExp, as well as between ConvertedCompYearly and JobSatPoints_1.

```
## Write your code here
```

Summary

In this lab, you practiced essential skills in correlation analysis by:

- Examining the distribution of yearly compensation with histograms and box plots.
- Detecting and removing outliers from compensation data.
- Calculating correlations between key variables such as compensation, work experience, and job satisfaction.
- Visualizing relationships with scatter plots and heatmaps to gain insights into the associations between these features.

By following these steps, you have developed a solid foundation for analyzing relationships within the dataset.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 16 - Data Visualization

Data Visualization

Estimated time needed: 45 minutes

In this lab, you will focus on data visualization. The dataset will be provided through an RDBMS, and you will need to use SQL queries to extract the required data.

Objectives

After completing this lab, you will be able to:

- Visualize the distribution of data.
- Visualize the relationship between two features.
- Visualize composition and comparison of data.

Demo: How to work with database

Download the database file.

```
!wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv
```

Install and Import Necessary Python Libraries

Ensure that you have the required libraries installed to work with SQLite and Pandas:

```
!pip install pandas
!pip install matplotlib

import pandas as pd
import matplotlib.pyplot as plt
```

Read the CSV File into a Pandas DataFrame

Load the Stack Overflow survey data into a Pandas DataFrame:

```
# Read the CSV file
df = pd.read_csv('survey-data.csv')

# Display the first few rows of the data
df.head()
```

Create a SQLite Database and Insert the Data

Now, let's create a new SQLite database (survey-data.sqlite) and insert the data from the DataFrame into a table using the sqlite3 library:

```
import sqlite3

# Create a connection to the SQLite database
conn = sqlite3.connect('survey-data.sqlite')

# Write the dataframe to the SQLite database
df.to_sql('main', conn, if_exists='replace', index=False)


# Close the connection
conn.close()
```

Verify the Data in the SQLite Database Verify that the data has been correctly inserted into the SQLite database by running a simple query:

```
# Reconnect to the SQLite database
conn = sqlite3.connect('survey-data.sqlite')

# Run a simple query to check the data
QUERY = "SELECT * FROM main LIMIT 5"
df_check = pd.read_sql_query(QUERY, conn)

# Display the results
print(df_check)
```

Demo: Running an SQL Query

Count the number of rows in the table named 'main'

```
QUERY = """
SELECT COUNT(*)
FROM main
"""
df = pd.read_sql_query(QUERY, conn)
df.head()
```

Demo: Listing All Tables

To view the names of all tables in the database:

```
QUERY = """
SELECT name as Table_Name FROM sqlite_master
WHERE type = 'table'
"""
pd.read_sql_query(QUERY, conn)
```

Demo: Running a Group By Query

For example, you can group data by a specific column, like Age, to get the count of respondents in each age group:

```
QUERY = """
SELECT Age, COUNT(*) as count
FROM main
GROUP BY Age
ORDER BY Age
"""
pd.read_sql_query(QUERY, conn)
```

Demo: Describing a table

Use this query to get the schema of a specific table, main in this case:

```
table_name = 'main'

QUERY = """
SELECT sql FROM sqlite_master
WHERE name= '{}'
""".format(table_name)

df = pd.read_sql_query(QUERY, conn)
print(df.iat[0,0])
```

Hands-on Lab

Visualizing the Distribution of Data

Histograms

Plot a histogram of CompTotal (Total Compensation).

```
## Write your code here
```

Box Plots

Plot a box plot of Age.

```
## Write your code here
```

Visualizing Relationships in Data

Scatter Plots

Create a scatter plot of Age and WorkExp.

```
## Write your code here
```

Bubble Plots

Create a bubble plot of TimeSearching and Frustration using the Age column as the bubble size.

```
## Write your code here
```

Visualizing Composition of Data

Pie Charts

Create a pie chart of the top 5 databases(DatabaseWantToWorkWith) that respondents wish to learn next year.

```
## Write your code here
```

Stacked Charts

Create a stacked bar chart of median TimeSearching and TimeAnswering for the age group 30 to 35.

```
## Write your code here
```

Visualizing Comparison of Data

Line Chart

Plot the median CompTotal for all ages from 45 to 60.

```
## Write your code here
```

Bar Chart

Create a horizontal bar chart using the MainBranch column.

```
## Write your code here
```

Summary

In this lab, you focused on extracting and visualizing data from an RDBMS using SQL queries and SQLite. You applied various visualization techniques, including:

- Histograms to display the distribution of CompTotal.
- Box plots to show the spread of ages.
- Scatter plots and bubble plots to explore relationships between variables like Age, WorkExp, TimeSearching and TimeAnswering.
- Pie charts and stacked charts to visualize the composition of data.
- Line charts and bar charts to compare data across categories.

Close the Database Connection

Once the lab is complete, ensure to close the database connection:

```
conn.close()
```

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 17 - Data Visualization (Histogram)

Histogram

Estimated time needed: 45 minutes

In this lab, you will focus on the visualization of data. The dataset will be provided through an RDBMS, and you will need to use SQL queries to extract the required data.

Objectives

In this lab, you will perform the following:

- Visualize the distribution of data using histograms.
- Visualize relationships between features.
- Explore data composition and comparisons.

Demo: Working with database

Download the database file.

```
!wget -O survey-data.sqlite https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/QR9YeprUYhOoLafzlLspAw
```

Install the required libraries and import them

```
!pip install pandas
```

```
!pip install matplotlib
```

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
```

Connect to the SQLite database

```
conn = sqlite3.connect('survey-data.sqlite')
```

Demo: Basic SQL queries

Demo 1: Count the number of rows in the table

```
QUERY = "SELECT COUNT(*) FROM main"
df = pd.read_sql_query(QUERY, conn)
print(df)
```

Demo 2: List all tables

```
QUERY = """
SELECT name as Table_Name
FROM sqlite_master
WHERE type = 'table'
"""
pd.read_sql_query(QUERY, conn)
```

Demo 3: Group data by age

```
QUERY = """
SELECT Age, COUNT(*) as count
FROM main
GROUP BY Age
ORDER BY Age
"""
df_age = pd.read_sql_query(QUERY, conn)
print(df_age)
```

Hands-on Lab: Visualizing Data with Histograms

1. Visualizing the distribution of data (Histograms)

1.1 Histogram of CompTotal (Total Compensation)

Objective: Plot a histogram of CompTotal to visualize the distribution of respondents' total compensation.

```
## Write your code here
```

1.2 Histogram of YearsCodePro (Years of Professional Coding Experience)

Objective: Plot a histogram of YearsCodePro to analyze the distribution of coding experience among respondents.

```
## Write your code here
```

2. Visualizing Relationships in Data

2.1 Histogram Comparison of CompTotal by Age Group

Objective: Use histograms to compare the distribution of CompTotal across different Age groups.

```
## Write your code here
```

2.2 Histogram of TimeSearching for Different Age Groups

Objective: Use histograms to explore the distribution of TimeSearching (time spent searching for information) for respondents across different age groups.

```
## Write your code here
```

3. Visualizing the Composition of Data

3.1 Histogram of Most Desired Databases (DatabaseWantToWorkWith)

Objective: Visualize the most desired databases for future learning using a histogram of the top 5 databases.

```
## Write your code here
```

3.2 Histogram of Preferred Work Locations (RemoteWork)

Objective: Use a histogram to explore the distribution of preferred work arrangements (remote work).

```
## Write your code here
```

4. Visualizing Comparison of Data

4.1 Histogram of Median CompTotal for Ages 45 to 60

Objective: Plot the histogram for CompTotal within the age group 45 to 60 to analyze compensation distribution among mid-career respondents.

```
## Write your code here
```

4.2 Histogram of Job Satisfaction (JobSat) by YearsCodePro

Objective: Plot the histogram for JobSat scores based on respondents' years of professional coding experience.

```
## Write your code here
```

Final step: Close the database connection

Once you've completed the lab, make sure to close the connection to the SQLite database:

```
conn.close()
```

Summary

In this lab, you used histograms to visualize various aspects of the dataset, focusing on:

- Distribution of compensation, coding experience, and work hours.
- Relationships in compensation across age groups and work status.
- Composition of data by desired databases and work environments.
- Comparisons of job satisfaction across years of experience.

Histograms helped reveal patterns and distributions in the data, enhancing your understanding of developer demographics and preferences.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 18 - Box Plots

Box Plots

Estimated time needed: 45 minutes

In this lab, you will focus on the visualization of data. The dataset will be provided through an RDBMS, and you will need to use SQL queries to extract the required data.

Objectives

In this lab you will perform the following:

- Visualize the distribution of data.
- Visualize the relationship between two features.
- Visualize data composition and comparisons using box plots.

Setup: Connecting to the Database

1. Download the Database File

```
!wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/QR9YeprUYhOoLafzlLspAw/survey-results-public.
```

2. Connect to the Database

Install the needed libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt

# Connect to the SQLite database
conn = sqlite3.connect('survey-results-public.sqlite')
```

Demo: Basic SQL Queries

Demo 1: Count the Number of Rows in the Table

```
QUERY = "SELECT COUNT(*) FROM main"
df = pd.read_sql_query(QUERY, conn)
print(df)
```

Demo 2: List All Tables

```
QUERY = """
SELECT name as Table_Name
FROM sqlite_master
WHERE type = 'table'
"""
pd.read_sql_query(QUERY, conn)
```

Demo 3: Group Data by Age

```
QUERY = """
SELECT Age, COUNT(*) as count
FROM main
GROUP BY Age
ORDER BY Age
"""
df_age = pd.read_sql_query(QUERY, conn)
print(df_age)
```

Visualizing Data

Task 1: Visualizing the Distribution of Data

1. Box Plot of CompTotal (Total Compensation)

Use a box plot to analyze the distribution and outliers in total compensation.

```
# your code goes here
```

2. Box Plot of Age (converted to numeric values)

Convert the Age column into numerical values and visualize the distribution.

```
# your code goes here
```

Task 2: Visualizing Relationships in Data

1. Box Plot of CompTotal Grouped by Age Groups:

Visualize the distribution of compensation across different age groups.

```
# your code goes here
```

2. Box Plot of CompTotal Grouped by Job Satisfaction (JobSatPoints_6):

Examine how compensation varies based on job satisfaction levels.

```
# your code goes here
```

Task 3: Visualizing the Composition of Data

1. Box Plot of ConvertedCompYearly for the Top 5 Developer Types:

Analyze compensation across the top 5 developer roles.

```
# your code goes here
```

2. Box Plot of CompTotal for the Top 5 Countries:

Analyze compensation across respondents from the top 5 countries.

```
# your code goes here
```

Task 4: Visualizing Comparison of Data

1. Box Plot of CompTotal Across Employment Types:

Analyze compensation for different employment types.

```
# your code goes here
```

2. Box Plot of YearsCodePro by Job Satisfaction (JobSatPoints_6):

Examine the distribution of professional coding years by job satisfaction levels.

```
# your code goes here
```

Final Step: Close the Database Connection

After completing the lab, close the connection to the SQLite database:

```
conn.close()
```

Summary

In this lab, you used box plots to visualize various aspects of the dataset, focusing on:

- Visualize distributions of compensation and age.
- Explore relationships between compensation, job satisfaction, and professional coding experience.
- Analyze data composition across developer roles and countries.
- Compare compensation across employment types and satisfaction levels.

Box plots provided clear insights into the spread, outliers, and central tendencies of various features in the dataset.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 19 - Scatter Plots

Scatter Plot

Estimated time needed: 45 minutes

Overview

In this lab, you will focus on creating and interpreting scatter plots to visualize relationships between variables and trends in the dataset. The provided dataset will be directly loaded into a pandas DataFrame, and various scatter

plot-related visualizations will be created to explore developer trends, compensation, and preferences.

Objectives

In this lab, you will:

- Create and analyze scatter plots to examine relationships between variables.
- Use scatter plots to identify trends and patterns in the dataset.
- Focus on visualizations centered on scatter plots for better data-driven insights.

Setup: Working with the Database

Install and import the required libraries

```
!pip install pandas
!pip install matplotlib

import pandas as pd
import matplotlib.pyplot as plt
```

Step 1: Load the dataset

```
file_path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/survey-data.csv

df = pd.read_csv(file_path)
```

Task 1: Exploring Relationships with Scatter Plots

1. Scatter Plot for Age vs. Job Satisfaction

Visualize the relationship between respondents' age (Age) and job satisfaction (JobSatPoints_6). Use this plot to identify any patterns or trends.

```
## Write your code here
```

2. Scatter Plot for Compensation vs. Job Satisfaction

Explore the relationship between yearly compensation (ConvertedCompYearly) and job satisfaction (JobSatPoints_6) using a scatter plot.

```
## Write your code here
```

Task 2: Enhancing Scatter Plots

1. Scatter Plot with Trend Line for Age vs. Job Satisfaction

Add a regression line to the scatter plot of Age vs. JobSatPoints_6 to highlight trends in the data.

```
## Write your code here
```

2. Scatter Plot for Age vs. Work Experience

Visualize the relationship between Age (Age) and Work Experience (YearsCodePro) using a scatter plot.

```
## Write your code here
```

Task 3: Combining Scatter Plots with Additional Features

1. Bubble Plot of Compensation vs. Job Satisfaction with Age as Bubble Size

Create a bubble plot to explore the relationship between yearly compensation (ConvertedCompYearly) and job satisfaction (JobSatPoints_6), with bubble size representing age.

```
## Write your code here
```

2. Scatter Plot for Popular Programming Languages by Job Satisfaction

Visualize the popularity of programming languages (LanguageHaveWorkedWith) against job satisfaction using a scatter plot. Use points to represent satisfaction levels for each language.

```
## Write your code here
```

Task 4: Scatter Plot Comparisons Across Groups

1. Scatter Plot for Compensation vs. Job Satisfaction by Employment Type

Visualize the relationship between yearly compensation (ConvertedCompYearly) and job satisfaction (JobSatPoints_6), categorized by employment type (Employment). Use color coding or markers to differentiate between employment types.

```
## Write your code here
```

2. Scatter Plot for Work Experience vs. Age Group by Country

Compare work experience (YearsCodePro) across different age groups (Age) and countries (Country). Use colors to represent different countries and markers for age groups.

```
## Write your code here
```

Final Step: Review

With these scatter plots, you will have analyzed data relationships across multiple dimensions, including compensation, job satisfaction, employment types, and demographics, to uncover meaningful trends in the developer community.

Summary

After completing this lab, you will be able to:

- Analyze how numerical variables relate across specific groups, such as employment types and countries.
- Use scatter plots effectively to represent multiple variables with color, size, and markers.
- Gain insights into compensation, satisfaction, and demographic trends using advanced scatter plot techniques.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 20 - Bubble Plots

Bubble Plots

Estimated time needed: 30 minutes

In this lab, you will focus on visualizing data.

The dataset will be directly loaded into pandas for analysis and visualization.

You will use various visualization techniques to explore the data and uncover key trends.

Objectives

In this lab, you will perform the following:

- Visualize the distribution of data.
- Visualize the relationship between two data features.
- Visualize composition of data.
- Visualize comparison of data.

Setup: Working with the Database

Install and import the needed libraries

```
!pip install pandas
!pip install matplotlib
```

```
import pandas as pd
import matplotlib.pyplot as plt
```

Download and connect to the database file containing survey data.

To start, download and load the dataset into a pandas DataFrame.

```
# Step 1: Download the dataset
!wget -O survey-data.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/sur

# Load the data
df = pd.read_csv("survey-data.csv")

# Display the first few rows of the data to understand its structure
df.head()
```

Task 1: Exploring Data Distributions Using Bubble Plots

1. Bubble Plot for Age vs. Frequency of Participation

- Visualize the relationship between respondents' age and their participation frequency (SOPartFreq) using a bubble plot.
- Use the size of the bubbles to represent their job satisfaction (JobSat).

```
##Write your code here
```

2. Bubble Plot for Compensation vs. Job Satisfaction

-Visualize the relationship between yearly compensation (ConvertedCompYearly) and job satisfaction (JobSat).

- Use the size of the bubbles to represent respondents' age.

```
##Write your code here
```

Task 2: Analyzing Relationships Using Bubble Plots

1. Bubble Plot of Technology Preferences by Age

- Visualize the popularity of programming languages respondents have worked with (LanguageHaveWorkedWith) across age groups.
- Use bubble size to represent the frequency of each language.

```
##Write your code here
```

2. Bubble Plot for Preferred Databases vs. Job Satisfaction

- Explore the relationship between preferred databases (DatabaseWantToWorkWith) and job satisfaction.
- Use bubble size to indicate the number of respondents for each database.

```
##Write your code here
```

Task 3: Comparing Data Using Bubble Plots

1. Bubble Plot for Compensation Across Developer Roles

- Visualize compensation (ConvertedCompYearly) across different developer roles (DevType).
- Use bubble size to represent job satisfaction.

```
##Write your code here
```

2. Bubble Plot for Collaboration Tools by Age

- Visualize the relationship between the collaboration tools used (NEWCollabToolsHaveWorkedWith) and age groups.
- Use bubble size to represent the frequency of tool usage.

```
##Write your code here
```

Task 4: Visualizing Technology Trends Using Bubble Plots

1. Bubble Plot for Preferred Web Frameworks vs. Job Satisfaction

- Explore the relationship between preferred web frameworks (WebframeWantToWorkWith) and job satisfaction.
- Use bubble size to represent the number of respondents.

```
##Write your code here
```

2. Bubble Plot for Admired Technologies Across Countries

- Visualize the distribution of admired technologies (LanguageAdmired) across different countries (Country).
- Use bubble size to represent the frequency of admiration.

```
##Write your code here
```

Final Step: Review

After completing the lab, you will have extensively used bubble plots to gain insights into developer community preferences, demographics, compensation trends, and job satisfaction.

Summary

After completing this lab, you will be able to:

- Create and interpret bubble plots to analyze relationships and compositions within datasets.
- Use bubble plots to explore developer preferences, compensation trends, and satisfaction levels.
- Apply bubble plots to visualize complex relationships involving multiple dimensions effectively.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-29 1.2 Madhusudhan Moole Updated lab

2024-10-16 1.1 Madhusudhan Moole Updated lab

2024-10-15 1.0 Raghul Ramesh Created lab

--!>

Lab 21 - Pie Charts

Pie Charts

Estimated time needed: 30 minutes

- In this lab, you will focus on visualizing data.
- The provided dataset will be loaded into pandas for analysis.
- Various pie charts will be created to:

- Analyze developer preferences.

- Identify technology usage trends.

- The lab aims to provide insights into key variables using visual representations.

Objectives

In this lab you will perform the following:

- Visualize the distribution of data.

- Visualize the relationship between two features.
- Visualize composition of data.
- Visualize comparison of data.

Setup: Downloading and Loading the Data

Install the libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

Download and Load the Data

To start, download and load the dataset into a pandas DataFrame.

```
# Step 1: Download the dataset
!wget -O survey-data.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/sur

# Step 2: Import necessary libraries and load the dataset
import pandas as pd
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv("survey-data.csv")

# Display the first few rows to understand the structure of the data
df.head()
```

Task 1: Visualizing Data Composition with Pie Charts

1.1 Create a Pie Chart of the Top 5 Databases Respondents Want to Work With

In the survey data, the DatabaseWantToWorkWith column lists the databases that respondents wish to work with. Let's visualize the top 5 most-desired databases in a pie chart.

```
##Write your code here
```

The DevType column lists the developer types for respondents. We'll examine the distribution by showing the top 5 developer roles in a pie chart.

```
##Write your code here
```

1.3 Create a pie chart for the operating systems used by respondents for professional use

The OpSysProfessional use column shows the operating systems developers use professionally. Let's visualize the distribution of the top operating systems in a pie chart.

```
##Write your code here
```

Task 2: Additional Visualizations and Comparisons

2.1 Pie Chart for Top 5 Programming Languages Respondents Have Worked With

The LanguageHaveWorkedWith column contains the programming languages that respondents have experience with. We'll plot a pie chart to display the composition of the top 5 languages.

```
##Write your code here
```

2.2 Pie Chart for Top Collaboration Tools used in Professional Use

Using the NEWCollabToolsHaveWorkedWith column, we'll identify and visualize the top collaboration tools respondents use in their professional work.

```
##Write your code here
```

Task 3: Analyzing and Interpreting Composition

In this task, you will create additional pie charts to analyze specific aspects of the survey data. Use pandas and matplotlib to complete each task and interpret the findings.

3.1 Pie Chart of Respondents Most Admired Programming Languages

The LanguageAdmired column lists the programming languages respondents admire most. Create a pie chart to visualize the top 5 admired languages.

```
##Write your code here
```

3.2 Pie Chart of Tools Used for AI Development

Using the AIToolCurrently Using column, create a pie chart to visualize the top 5 tools developers are currently using for AI development.

```
##Write your code here
```

3.3 Pie Chart for Preferred Web Frameworks

The WebframeWantToWorkWith column includes web frameworks that respondents are interested in working with. Visualize the top 5 frameworks in a pie chart.

```
##Write your code here
```

3.4 Pie Chart for Most Desired Embedded Technologies

Using the EmbeddedWantToWorkWith column, create a pie chart to show the top 5 most desired embedded technologies that respondents wish to work with.

```
##Write your code here
```

Summary

After completing this lab, you will be able to:

- Create pie charts to visualize developer preferences across databases, programming languages, AI tools, and cloud platforms.
- Identify trends in technology usage, role distribution, and tool adoption through pie charts.
- Analyze and compare data composition across various categories to gain insights into developer preferences.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

Lab 22 - Stacked Charts

Stacked Charts

Estimated time needed: 45 minutes

In this lab, you will focus on visualizing data specifically using stacked charts. You will use SQL queries to extract the necessary data and apply stacked charts to analyze the composition and comparison within the data.

Objectives

In this lab, you will perform the following:

- Visualize the composition of data using stacked charts.
- Compare multiple variables across different categories using stacked charts.
- Analyze trends within stacked chart visualizations.

Setup: Downloading and Loading the Data

Install the libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

Download and Load the Data

To start, download and load the dataset into a pandas DataFrame.

Step 1: Download the dataset

```
!wget -O survey-data.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/sur
```

Step 2: Import necessary libraries and load the dataset

```
import pandas as pd
import matplotlib.pyplot as plt
```

Load the data

```
df = pd.read_csv("survey-data.csv")
```

Display the first few rows of the data to understand its structure

```
df.head()
```

Task 1: Stacked Chart for Composition of Job Satisfaction Across Age Groups

1. Stacked Chart of Median JobSatPoints6 and JobSatPoints7 for Different Age Groups

Visualize the composition of job satisfaction scores (JobSatPoints6 and JobSatPoints7) across various age groups. This will help in understanding the breakdown of satisfaction levels across different demographics.

```
##Write your code here
```

Stacked Chart of JobSatPoints6 and JobSatPoints7 for Employment Status

Create a stacked chart to compare job satisfaction (JobSatPoints6 and JobSatPoints7) across different employment statuses. This will show how satisfaction varies by employment type.

```
##Write your code here
```

Task 2: Stacked Chart for Compensation and Job Satisfaction by Age Group

This stacked chart visualizes the composition of compensation (ConvertedCompYearly) and job satisfaction (JobSatPoints_6) specifically for respondents aged 30-35.

```
##Write your code here
```

Stacked Chart of Median Compensation and Job Satisfaction Across Age Group

Compare the median compensation and job satisfaction metrics across different age groups. This helps visualize how compensation and satisfaction levels differ by age.

```
##Write your code here
```

Task 3: Comparing Data Using Stacked Charts

1. Stacked Chart of Preferred Databases by Age Group

Visualize the top databases that respondents from different age groups wish to learn. Create a stacked chart to show the proportion of each database in each age group.

```
##Write your code here
```

2. Stacked Chart of Employment Type by Job Satisfaction

Analyze the distribution of employment types within each job satisfaction level using a stacked chart. This will provide insights into how employment types are distributed across various satisfaction ratings.

```
##Write your code here
```

Task 4: Exploring Technology Preferences Using Stacked Charts

1. Stacked Chart for Preferred Programming Languages by Age Group

Analyze how programming language preferences (LanguageAdmired) vary across age groups.

```
##Write your code here
```

2. Stacked Chart for Technology Adoption by Employment Type

Explore how admired platforms (PlatformAdmired) differ across employment types (e.g., full-time, freelance)

```
##Write your code here
```

Final Step: Review

In this lab, you focused on using stacked charts to understand the composition and comparison within the dataset. Stacked charts provided insights into job satisfaction, compensation, and preferred databases across age groups and employment types.

Summary

After completing this lab, you will be able to:

- Use stacked charts to analyze the composition of data across categories, such as job satisfaction and compensation by age group.
- Compare data across different dimensions using stacked charts, enhancing your ability to communicate complex relationships in the data.
- Visualize distributions across multiple categories, such as employment type by satisfaction, to gain a deeper understanding of patterns within the dataset.

Author:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-28 1.2 Madhusudhan Moole Updated lab

2024-10-16 1.1 Madhusudhan Moole Updated lab

2024-10-15 1.0 Raghul Ramesh Created lab

--!>

Lab 23 - Line Charts

Line Charts

Estimated time needed: 30 minutes

In this lab, you will focus on using line charts to analyze trends over time and across different categories in a dataset.

Objectives

In this lab you will perform the following:

- Track trends in compensation across age groups and specific age ranges.
- Analyze job satisfaction trends based on experience level.
- Explore and interpret line charts to identify patterns and trends.

Setup: Working with the Database

Install the needed libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

Download and connect to the database file containing survey data.

To start, download and load the dataset into a pandas DataFrame.

Step 1: Download the dataset

```
!wget -O survey-data.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/sur
```

Step 2: Import necessary libraries and load the dataset

```
import pandas as pd
import matplotlib.pyplot as plt
```

Load the data

```
df = pd.read_csv("survey-data.csv")
```

Display the first few rows to understand the structure of the data

```
df.head()
```

Task 1: Trends in Compensation Over Age Groups

1. Line Chart of Median ConvertedCompYearly by Age Group

- Track how the median yearly compensation (ConvertedCompYearly) changes across different age groups.
- Use a line chart to visualize these trends.

```
## Write your code here
```

2. Line Chart of Median ConvertedCompYearly for Ages 25 to 45

For a closer look, plot a line chart focusing on the median compensation for respondents between ages 25 and 45.

```
## Write your code here
```

Task 2: Trends in Job Satisfaction by Experience Level

1. Line Chart of Job Satisfaction (JobSatPoints_6) by Experience Level

- Use a column that approximates experience level to analyze how job satisfaction changes with experience.
- If needed, substitute an available experience-related column for Experience.

```
## Write your code here
```

Task 3: Trends in Job Satisfaction and Compensation by Experience

1.Line Chart of Median ConvertedCompYearly Over Experience Level

- This line chart will track how median compensation (ConvertedCompYearly) changes with increasing experience.
- Use a column such as WorkExp or another relevant experience-related column.

```
## Write your code here
```

2.Line Chart of Job Satisfaction (JobSatPoints_6) Across Experience Levels

- Create a line chart to explore trends in job satisfaction (JobSatPoints_6) based on experience level.
- This chart will provide insight into how satisfaction correlates with experience over time

```
## Write your code here
```

Final Step: Review

In this lab, you focused on analyzing trends in compensation and job satisfaction, specifically exploring how these metrics change with age and experience levels using line charts.

Summary

In this lab, you explored essential data visualization techniques with a focus on analyzing trends using line charts. You learned to:

- Visualize the distribution of compensation across age groups to understand salary trends.
- Track changes in median compensation over various experience levels, identifying how earnings progress with experience.
- Examine trends in job satisfaction by experience, revealing how satisfaction varies throughout a developer's career.

These analyses allow for a deeper understanding of how factors like age and experience influence job satisfaction and compensation. By using line charts, you gained insights into continuous data patterns, which are invaluable for interpreting professional trends in the developer community.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika

<!--

Change Log

Date (YYYY-MM-DD) Version Changed By Change Description

2024-10-28 1.2 Madhusudhan Moole Updated lab

2024-10-16 1.1 Madhusudhan Moole Updated lab

2024-10-15 1.0 Raghul Ramesh Created lab

--!>

Lab 24 - Bar Charts

Bar Charts

Estimated time needed: 30 minutes

In this lab, you will focus on visualizing data.

The dataset will be provided to you in the form of an RDBMS.

You will use SQL queries to extract the necessary data.

Objectives

In this lab you will perform the following:

- Visualize the distribution of data
- Visualize the relationship between two features
- Visualize the composition of data
- Visualize comparison of data

Setup: Working with the Database

Install the needed libraries

```
!pip install pandas
```

```
!pip install matplotlib
```

Download and connect to the database file containing survey data.

To start, download and load the dataset into a pandas DataFrame.

```
# Step 1: Download the dataset
!wget -O survey-data.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/n01PQ9pSmiRX6520flujwQ/sur

# Step 2: Import necessary libraries and load the dataset
import pandas as pd
import matplotlib.pyplot as plt

# Load the data
df = pd.read_csv("survey-data.csv")

# Display the first few rows to understand the structure of the data
df.head()
```

Task 1: Visualizing Data Distributions

1. Histogram of ConvertedCompYearly

Visualize the distribution of yearly compensation (ConvertedCompYearly) using a histogram.

```
## Write your code here
```

2. Box Plot of Age

Since Age is categorical in the dataset, convert it to numerical values for a box plot.

```
## Write your code here
```

Task 2: Visualizing Relationships in Data

1. Scatter Plot of Age_numeric and ConvertedCompYearly

Explore the relationship between age and compensation.

```
## Write your code here
```

2. Bubble Plot of ConvertedCompYearly and JobSatPoints6 with Agenumeric as Bubble Size

Explore how compensation and job satisfaction are related, with age as the bubble size.

```
## Write your code here
```

Task 3: Visualizing Composition of Data with Bar Charts

1. Horizontal Bar Chart of MainBranch Distribution

Visualize the distribution of respondents' primary roles to understand their professional focus.

```
## Write your code here
```

2. Vertical Bar Chart of Top 5 Programming Languages Respondents Want to Work With

Identify the most desired programming languages based on LanguageWantToWorkWith.

```
## Write your code here
```

3. Stacked Bar Chart of Median JobSatPoints6 and JobSatPoints7 by Age Group

Compare job satisfaction metrics across different age groups with a stacked bar chart.

```
## Write your code here
```

4. Bar Chart of Database Popularity (DatabaseHaveWorkedWith)

Identify the most commonly used databases among respondents by visualizing DatabaseHaveWorkedWith.

```
## Write your code here
```

Task 4: Visualizing Comparison of Data with Bar Charts

1. Grouped Bar Chart of Median ConvertedCompYearly for Different Age Groups

Compare median compensation across multiple age groups with a grouped bar chart.

```
## Write your code here
```

2. Bar Chart of Respondent Count by Country

Show the distribution of respondents by country to see which regions are most represented.

```
## Write your code here
```

Final Step: Review

This lab demonstrates how to create and interpret different types of bar charts, allowing you to analyze the composition, comparison, and distribution of categorical data in the Stack Overflow dataset, including main professional branches, programming language preferences, and compensation by age group. Bar charts effectively compare counts and median values across various categories.

Summary

After completing this lab, you will be able to:

- Create a horizontal bar chart to visualize the distribution of respondents' primary roles, helping to understand their professional focus.
- Develop a vertical bar chart to identify the most desired programming languages based on the LanguageWantToWorkWith variable.
- Use a stacked bar chart to compare job satisfaction metrics across different age groups.
- Create a bar chart to visualize the most commonly used databases among respondents using the DatabaseHaveWorkedWith variable.

Authors:

Ayushi Jain

Other Contributors:

- Rav Ahuja
- Lakshmi Holla
- Malika